

An Approach of Generating Test Requirements for Agile Software Development

Saurabh Tiwari

Indian Institute of Information Technology,
Design and Manufacturing, Jabalpur, INDIA
saurabh.tiwari@iiitdmj.ac.in

Atul Gupta

Indian Institute of Information Technology,
Design and Manufacturing, Jabalpur, INDIA
atul@iiitdmj.ac.in

ABSTRACT

Test driven development, an agile programming approach, has been advocated as a promising approach for developing quality code. This approach aims at developing minimal, non-redundant code accepting only one test case at a time. However, not much has been deliberated in the literature on how to systematically generate test cases for incremental coding. In this paper, we propose an approach of generating test requirements from use case specification for test driven development. The approach consists of first constructing actor-oriented activity diagrams from use case specifications, and then generating systematic, prioritized test sequences from these activity diagrams for developing code incrementally. We demonstrate the applicability of the proposed approach using an illustrative example and discuss other related issues. We also present a comparative study of the effectiveness of test sequences generated from our approach with some other existing use case based test case generation approaches.

Keywords

Use case specification, UML activity diagram, test requirements, prioritization, agile software development

1. INTRODUCTION

Agile development is aligned with the view to face the challenges of an increasingly volatile marketplace, changing requirements, customer involvement, priorities and shorter deadlines [1]. All these factors have led to the adoption of practices which can foster a more lightweight software development environment. Some of the popular agile methodologies include Extreme Programming (XP), Dynamic Systems Development Method (DSDM), Test Driven Development (TDD), Feature Driven Development (FDD), and Scrum [14]. In spite of having a common vision and core values, the agile methods tends to be unique in their specific approaches.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org
ISEC'15, Feb 18 - 20, 2015, Bangalore, Karnataka, India
Copyright 2015 ACM 978-1-4503-3432-7/15/02...\$15.00
<http://dx.doi.org/10.1145/2723742.2723761>

TDD, an agile code development approach, does away with a few nuances of the testing by entwining the coding and testing phase. In TDD, we typically obtain test cases from the requirement specification and then proceed iteratively with the code development process. We pick a test case and develop a minimal and non-redundant code for it. Then, we modify the code so that it gets executed successfully on the next selected test case. We always ensure that in each successive iterations, the integrity of modified code for previous test cases is maintained. TDD indicates two challenges - identify test cases and prioritize them for incremental code generation.

Though many approaches for generating test requirements from specification have been proposed [3][5][13][16][25], most of them target testing as an after effect rather than generating code incrementally. What we need here is to find suitable test cases that can guide the code development process incrementally with minimum up front designing. At the same time, the approach should be able to cope up with release planning and efficient change management.

In this paper, we propose an approach of generating test requirements from the use case specification by identifying and integrating various interactions of an actor with the system. This method of transforming use case specification into activity diagram captures functional dependencies - the inter-dependencies between various activities pertaining to different use cases. We then synthesize the test cases from the activity diagram. Next, is test case prioritization which aids in deciding the order of precedence of the test cases while we iterate over the TDD approach. This is imperative as there will be certain combination of the inputs which has relatively higher probability of occurrence. Consequently, the corresponding test cases attain greater importance for the system. Many criteria and parameters exist which guides the developer in this process [19]. As TDD aims at approaching the coding phase in an iterative and incremental manner so we choose test cases sequentially, in the order of increasing complexities.

We demonstrate the applicability of the proposed approach using an illustrative example of FRIEND, an accident management system. The results suggested that the approach generates systematic and prioritized test sequences from the integrated activity diagram per actor - suitable for the agile-based software development process. Arguably, our approach also helps in (1) merging the use cases to reduce testing effort - by reducing the number of activity diagrams (2) prioritizing tests by considering the customer's discretion.

The rest of the paper is organized as follows. Section 2 the

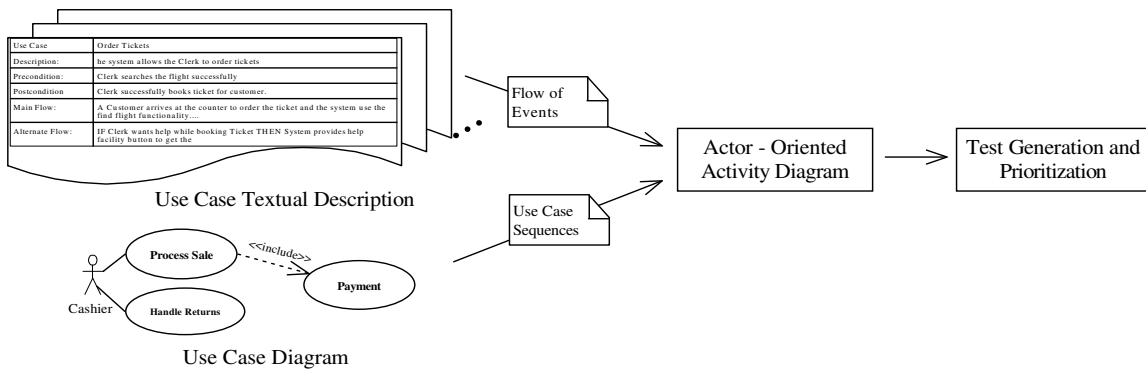


Figure 1: Overview of the proposed approach

proposed approach and illustrations using some examples. Section 3 presents the analysis of various use case based test generation approaches. We discuss the applicability of the proposed approach and other related issues in Section 4, followed by the related work in Section 5. In Section 6, we conclude our work and suggest future directions.

2. THE APPROACH

In this section, we describe our approach of generating test requirements for agile software development which involves transforming use case requirements into activity diagrams. We generate activity diagram by identifying various interaction of an actor with the system and the flow of events sequenced within the use cases. The generated activity diagram is then used to generate the test cases along with their prioritization. Overview of the proposed approach is shown in Figure 1.

Basic Steps

The approach counts two steps, which are as follows.

1. Transformation of use case specifications into one activity diagram per actor
2. Derivation of prioritized test suite from the activity diagram

The steps of the proposed approach are illustrated through a running example of FRIEND (First Responder Interactive Emergency Navigational Database) system [2]. It consists of seven use cases - *ReportEmergency*, *OpenIncident*, *CreateIncident*, *ManageIncident*, *CloseIncident*, *ConnectionDown* and *AllocateResources*; and has two actors - *Field Officer* and *Dispatcher*. *Field Officer* has access to a wireless device that enables them to interact with the control station. The actor *Dispatcher*, sitting in the control station, in turn can see the current status of all its resources and can respond to an incident by assigning resources along with necessary instructions. The use case diagram for the same is shown in Figure 2 and the use case textual description of the *Report Emergency*, *Open Incident* and *Allocate Resource* use cases are shown in Table 1, Table 2 and Table 3 respectively.

2.1 Transformation into one Activity Diagram per Actor

Let us consider some basic assumptions before generating actor-oriented activity diagrams - a thorough and detailed specification of the use case is available; and the `<<include>>`

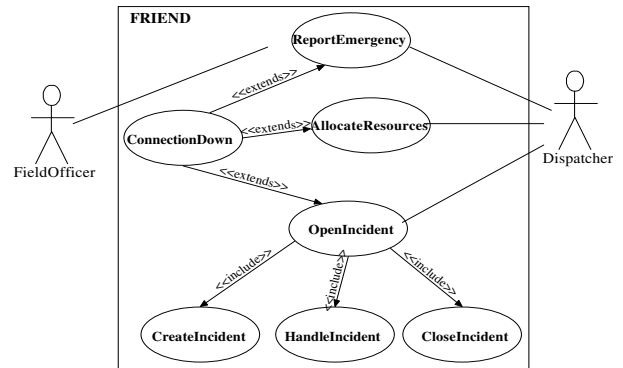


Figure 2: Use case diagram of a FRIEND System

and `<<extend>>` relations have been used while specifying requirements in the use cases.

We assume that the software functional requirements are specified using UML use case specification template [9]. This template consists of several fields such as use case name, goal, actors, main flows, sub flows, alternate flows and exceptional flows. The sub flows are the part of either main flow or another flow, so that it can be reused within the use case. It is created in a separate paragraph and is the optional part of the use case specification [20]. The actor field in a template specifies two distinct type of actors - primary and secondary actors. The primary actor is directly associated with the use case to perform the desired functionalities and, the secondary actor provides the supporting role in accomplishing those functionalities. For example, in an online shopping facility, there will be three actors involved, namely the administrator, customer and the vendor. We can easily see that the intended goals of the actors are different. The responsibility of the administrator is the proper functioning and maintaining the service while others have to avails the services offered by the system. When two actors work in collaboration, the use cases for each of the two actors will account for the responses generated by one actor in response to the stimulus provided by another actor.

Use cases are related to each other by means of relations such as `<<include>>` and `<<extend>>`, but there may also have sequential dependencies between the use cases [3][21]. There is no mechanism in the use cases for specifying the dependencies amongst the flow of events. The dependencies such as 'event E_1 requires event E_2 to be previously executed'; and the sequential dependencies such as 'use case UC_1 requires that use case UC_2 has been previously exe-

Table 1: Report Emergency Use case

Use Case Name (UC_{id})	UC1	ReportEmergency
Actors (AC_{id})		FieldOfficer, Dispatcher
Goal		The Field officer reports the emergency to the Dispatcher by activating the function on the terminal.
Pre-Condition(s) (Pr_{id})	Pr1.1	The FieldOfficer is logged into FRIEND.
Main Flow (M_{id})	M1.1	FieldOfficer activates the 'Report Emergency' function of FRIEND terminal
	M1.2	FRIEND responds by presenting a form to the FieldOfficer.
	M1.3	FieldOfficer clicks on open form button and describes possible responses to the emergency situation [S1.1][S1.2]
	M1.4	FieldOfficer clicks on submit button
	M1.5	The Dispatcher reviews the submitted information [S1.3][S1.4][S1.5]
	M1.6	Dispatcher submits the final report and then FRIEND displays the acknowledgment and response to the FieldOfficer.
	M1.7	Unable to make a connection [A1.1]
Sub Flow (S_{id})	S1.1	FieldOfficer select the emergency level, type, location, and brief description of the situation.
	S1.2	The FieldOfficer also describes possible responses to the emergency situation
	S1.3	Creates an Incident in the database
	S1.4	Dispatcher selects a response
	S1.5	Dispatcher submits the response to the FieldOfficer
Alternate Flow (A_{id})	A1.1	Extend use case ConnectionDown [E1.1]
Exceptional Flow (E_{id})	E1.1	Connection lost between the FieldOfficer and Dispatcher. Then, FRIEND responds the FieldOfficer and Dispatcher to try again
Post Condition(s) (Po_{id})	Po1.1	The FieldOfficer has received an acknowledgment and the selected response from the Dispatcher

Table 2: Open Incident Use case

Use Case Name (UC_{id})	UC2	OpenIncident
Actors (AC_{id})		Dispatcher
Goal		The Dispatcher manages the incident on the basis of emergency reported by the FieldOfficer
Pre-Condition(s) (Pr_{id})	Pr1.1	Dispatcher receives the information from the field officer.
Main Flow (M_{id})	M1.1	Dispatcher select a response sent by field officer [E1.1]
	M1.2	If Information is present, Dispatcher Include Create Incident UC [A1.1]
	M1.3	On the basis of priority incident (low or high), Dispatcher allocate the resources to the incident [A1.2][A1.3]
	M1.4	Dispatcher Include Manage Incident UC [S1.1][S1.2][S1.3]
	M1.5	Dispatcher close the incident Include Close Incident UC
Sub Flows (S_{id})	S1.1	Dispatcher receives reports and allocates resources.
	S1.2	Dispatcher coordinates the resources.
	S1.3	Dispatcher document the incident activity (about the allocation of resources and the action taken)
Alternate Flows (A_{id})	A1.1	If didn't receive the information by the field officer, or in the database, go back to the entry condition, remain in the idle mode.
	A1.2	For the low priority incident, the dispatcher allocates the resources of its own and the resource allocated is then committed to the specified incident.
	A1.3	For the highest priority incident, the dispatcher notifies the chief for the allocation of resource to the specified incident.
Exceptional Flows (E_{id})	E1.1	Unable to select the response (restart the database or try again)
	E1.2	Unable to connect, Connection lost between the FieldOfficer and Dispatcher
Post Condition(s) (Po_{id})	Po1.1	Dispatcher opens an Incident.

Table 3: Allocate Resources Use case

Use Case Name (UC_{id})	UC3	AllocateResources
Actors (AC_{id})		Dispatcher
Goal		The Dispatcher allocates the resources on the basis of Incident
Pre-Condition(s)	Pr1.1	The Dispatcher opens an Incident.
Main Flow (M_{id})	M1.1	Dispatcher selects an Emergency Incident
	M1.2	Dispatcher allocates the resources to the incident.
	M1.3	Dispatcher sends the resources information to the FieldOfficer
Alternate Flows (A_{id})	-	-
Exceptional Flows (E_{id})	-	-
Post Condition(s) (Po_{id})	Po1.1	Resources are assigned to the Incident.
	Po1.2	Resources receive notice about their new assignment.
	Po1.3	FieldOfficer in charge of the Incident receives notice about the new Resources.

cuted' cannot be expressed in the use case models either [11].

The sequential dependencies between the use cases can be represented by modeling the specifications into the activity diagram for each actor associated with the system. This facilitates the identification and visualization of these inter-use case dependencies. The vertices stand for the use cases and the edges corresponds to the sequential dependencies between the use cases. The <<include>> and <<extend>> use cases are represented by the nodes with subsequent edges as dotted. The sequence between the use cases can be generated by considering the pre- and post conditions of the use

case where precondition of a use case may be same as the postcondition of another use case. Therefore, there exists a sequence between the use cases irrespective of the relationships.

The use cases may also be grouped into swimlanes¹ according to their functionalities corresponding to each actor. Each swimlane is represented as actor-life history which de-

¹“The flow between the system and the actors within the use cases can be represented using swim lanes to separate the activities of different actors. This makes the user tasks, the division of work between user and the system, explicit” [18].

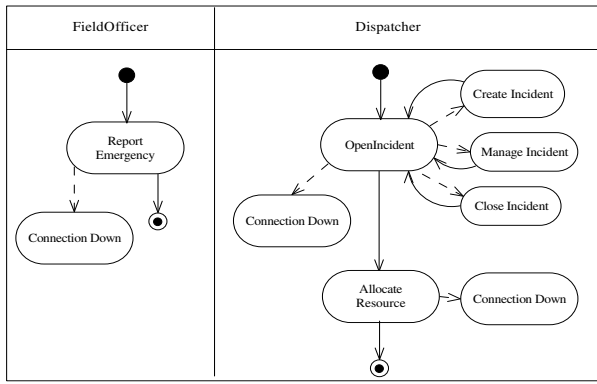


Figure 3: Sequencing among the use cases

scribe the life cycle of an actor in the system through all the functions performed by them or by their secondary actors, till the end of functionality [4]. Figure 3 shows the use case sequences generated for both the *Dispatcher* and *Field Officer*.

Next, we generate actor-oriented activity diagrams by identifying various interactions of an actor with the system and the events sequenced thereafter. This transformation of the actor-oriented activity diagrams from the use case models make use of the use case textual descriptions and the sequencing between the use cases corresponding to each actor.

The steps to generate the actor-oriented activity diagram are stated below.

1. Read the input list (i.e., use case sequences) containing the information about the use cases corresponding to each actor
2. Read the use case descriptions corresponding to each actor from the input list.
3. The transformation begins at first precondition. It is a initial node of the activity diagram.
4. Next, the first main flow id of the use case is considered as another node in the activity diagram.
5. Fetch the actor associated with each operation from the main flow.
6. Append the steps of the main flow into the activity diagram as a node (along with the associated actor)
7. Repeat steps 4 to 7 until all main flows are completed.
8. If the main flow of the use case have any sub flow, alternate flow, exceptional flow or the relations (`<<include>>` or `<<extend>>`) then
 - For each sub flow, the fork and join element of the activity diagram is use to model the actions after the main flow action. This information is extracted using the associated sub flow option stated in the use case template.
 - For each alternate flow, the guard condition (decision node) is use to model the actions after the main flow action. This information is alternative information therefore, the flow may return back to the main flow.

- For each exceptional flow, the error node is use to model the actions after the main flow action.
- For each relation, the use template has a predefined structure. For instance, the `<<include>>` relation is considered as a part of the main flow, while `<<extend>>` relation is a part of alternate flow.

9. The postcondition of the use case is the final node (successful completion) of the activity diagram corresponding to the actor.
10. The exceptional flow of the use case is the error or failure node of the activity diagram.
11. If use case has more than one actor then, create a swimlane and append it with the diagram.

The generated actor-oriented activity diagram corresponding to the *Dispatcher* is shown in Figure 4. First, we choose *Dispatcher* as an actor to create activity diagram. The *Dispatcher* is responsible for the functionality of both the use cases - *AllocateResources* and *OpenIncident*. The use case textual description for the *Dispatcher* is taken as input. According to the functionality, *OpenIncident* use case is performed prior to the *AllocateResources* use case. We found that these two use cases are sequentially dependent, whereas other use cases are related using `<<include>>` and `<<extend>>` relations. In Figure 4, the grey nodes represent the use cases included in the *OpenIncident* use case; the pink nodes are the flow of action events of *AllocateResources* use case, whereas the rest of nodes represent action events of *OpenIncident* use case performed by the *Dispatcher*.

Next, we choose *Field Officer* to generate activity diagram. The functionalities of a *ReportEmergency* use case was accomplished by two different actors - *Field Officer* and *Dispatcher*. *Field Officer* is a primary actor and *Dispatcher* is a secondary actor. Hence, the activity diagram for this case is implemented using the swimlane behavior where, the atomic operations from Pr1.1 to M1.4 are performed by the *Field Officer* and M1.5, M1.6 by the *Dispatcher*. The generated actor-oriented activity diagram is shown in Figure 5. In this Figure, the pink nodes represent the flow of action events performed by the secondary actor for the *ReportEmergency* use case, whereas the rest of nodes represent the sequence of actions performed by the *Field Officer* - a primary actor.

2.2 Test Case Generation and Prioritization

The essential part of the agile software development is the test case generation from the requirements in order to develop quick-quality code. Model based testing are suitable for generating the test requirements for agile projects by developing automated and user friendly tools [12]. There are opportunities to introduce model-based practices to increase test coverage especially when quality is essential; and hence this paper is an efforts towards the generation and prioritization of the test cases for TDD. We use the activity coverage and the path coverage test adequacy criteria to generate test cases from the activity diagram. Activity coverage requires that all activity nodes in the diagram be covered; and the path coverage requires that all paths in the diagram be covered [29]. Here, we assume that both the criteria consider a loop to be executed at most once. The test scenarios generated from both these criteria are then used to prioritize the

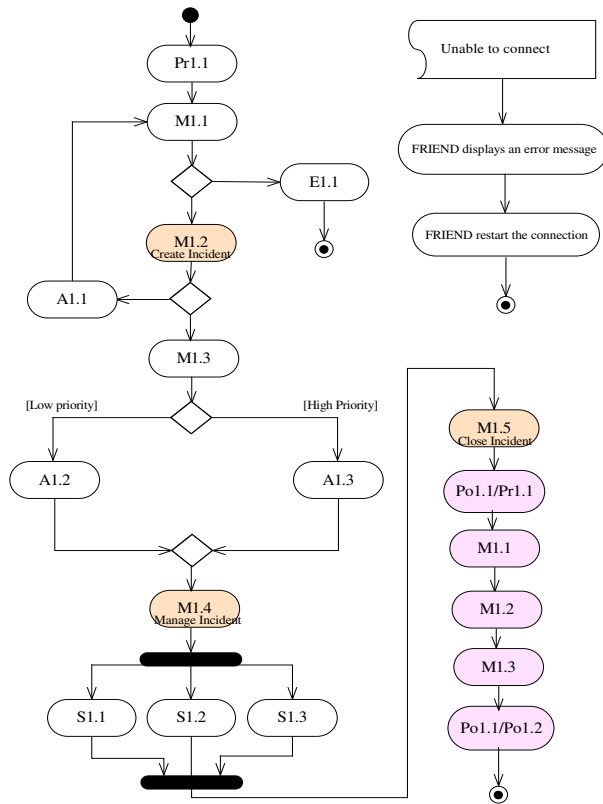


Figure 4: Activity diagram for the *Dispatcher*

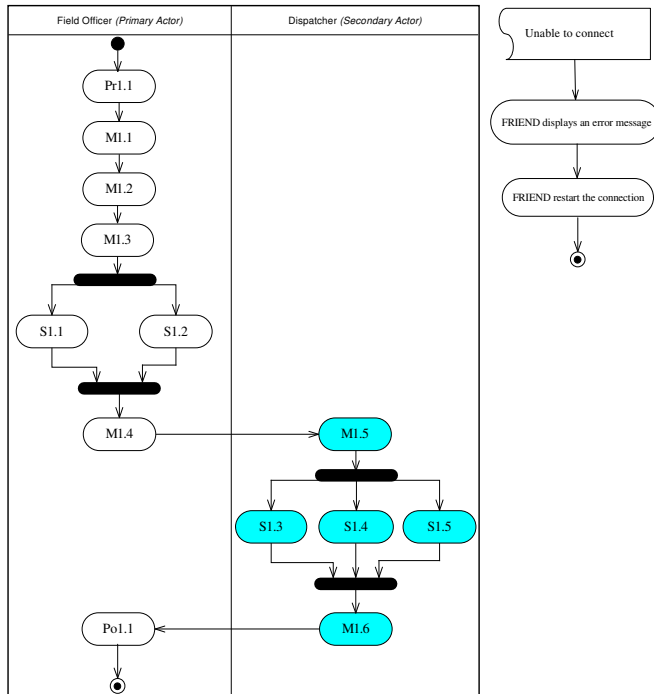


Figure 5: Activity diagram for the *Field Officer*

test sequences are per the customer's discretion. Table 4 shows test sequences generated from the activity diagrams of the *Field Officer* and the *Dispatcher*.

The test sequences generated are capable of testing the

common functionality of the use cases only once which reduces time and cost. Our approach generates the systematic, smaller to larger independent test sequences for each actor separately, add significant advantage over other techniques for incremental code development.

Now, the next step is to generate the prioritized test suite. Every possible path from initial start state to an end state represents a test scenario. On every path there are many inputs and corresponding to those inputs are the outputs. Each input - output pair represents a test case. The output of one such pair will serve as the input for the next test case and so on. We have to prioritize the test scenarios, also called the system usage scenarios or usage scenarios.

The priority field of the use case template is required now. In general, the use cases don't include the priority related information, but, the same can be assigned based on the customer's discretion. Of the many paths available from a *node n*, highest preference is given to the path with highest customer priority. The priority field of the first node of all the paths is compared and the one with the least numerical value is traversed first and so on. The reason being that lower numerical value represents a higher priority. The contention arises when the customer priority of two or more paths happens to be the same. To resolve this contention we make a call to the *decide_priority(*node,*a,*b)* routine. This routine explores all possible paths originating from the *node n* and returns the number of sub flows, alternate flows and main flows and their sum, encountered while traversing from the *node n* to the leaf node or the last node in case of a loop. A leaf node is a node that does not have any outgoing path to the new nodes. In case of a loop, we have to detect the loop and find the last node. By the term last node we mean the last node in loop which has not been already traversed. Suppose nodes A, B, C, D and E forms a loop ABCDEB. The last node for this loop will be node E as after node E we reach node B which has been already traversed.

The scheme for assigning priority is that the path with the lowest sum (the total number of sub flow, alternate flow and main flow) is traversed first. If this sum is equal for two paths, then we will give higher priority to that path which has lesser number of sub flows. If it is also equal then a higher priority is assigned to that path which has lesser number of alternate flows. If the contention is still not resolved, then we choose any path. The reason for above order is that to model a sub flow or any parallel or concurrent flow, threads have to be created. Synchronization and execution of threads will definitely be more complex vis-a-vis an alternate or main flow. Similarly, an alternate flow increases the branching or changes the control flow of the code. Hence, an alternate flow is relatively more complex than the main flow.

Let us assume that a *node n* has *m* nodes originating from it, namely *node*₁, *node*₂, ..., *node*_{*m*}. So we have *m* different paths from *node n*. Corresponding to each *node*_{*i*}, where 1 ≤ *i* ≤ *m*, a depth-first traversal (DFT) is implemented. While executing the DFT, we keep a record of the node already traversed in an array. And whenever a new node is to be added in this array, it is first compared with each entry of this array. If there is a match, then it indicates the presence of a loop in the activity diagram and the new node is not traversed again. In this manner we go on implementing the DFT recursively till we encounter a leaf node or the last node of a loop. The above process is implemented by

Table 4: Test sequences generated from the activity diagrams - FRIEND System

Test Sequences		
S.No.	Activity Diagram - <i>Dispatcher</i>	Activity Diagram - <i>Field Officer</i>
1	Pr1.1-M1.1	Pr1.1-M1.1
2	Pr1.1-M1.1-E1.1	Pr1.1-M1.1-M1.2
3	Pr1.1-M1.1-M1.2	Pr1.1-M1.1-M1.2-M1.3
4	Pr1.1-M1.1-M1.2-A1.1	Pr1.1-M1.1-M1.2-M1.3-S1.1:S1.2-JOIN
5	Pr1.1-M1.1-M1.2-A1.1-M1.1	Pr1.1-M1.1-M1.2-M1.3-S1.1:S1.2-JOIN-M1.4
6	Pr1.1-M1.1-M1.2-M1.3	Pr1.1-M1.1-M1.2-M1.3-S1.1:S1.2-JOIN-M1.4-M1.5
7	Pr1.1-M1.1-M1.2-M1.3-A1.2	...M1.2-M1.3-S1.1:S1.2-JOIN-M1.4-M1.5
8	Pr1.1-M1.1-M1.2-M1.3-A1.2-M1.4	...M1.2-M1.3-S1.1:S1.2-JOIN-M1.4-M1.5-S1.3:S1.4:S1.5-JOIN
9	Pr1.1-M1.1-M1.2-M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN	...M1.2-M1.3-S1.1:S1.2-JOIN-M1.4-M1.5-S1.3:S1.4:S1.5-JOIN-M1.6
10	...M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5	...M1.2-M1.3-S1.1:S1.2-JOIN-M1.4-M1.5-S1.3:S1.4:S1.5-JOIN-M1.6-Po1.1
11	...M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1	
12	...M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1	
13	...M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1-1.2	
14	...M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1-M1.2-M1.3	
15	...M1.3-A1.2-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1-M1.2-M1.3-Po1.2	
16	Pr1.1-M1.1-M1.2-M1.3-A1.3	
17	Pr1.1-M1.1-M1.2-M1.3-A1.3-M1.4	
18	Pr1.1-M1.1-M1.2-M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN	
19	...M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5	
20	...M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1	
21	...M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1	
22	...M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1-M1.2	
23	...M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1-M1.2-M1.3	
24	...M1.3-A1.3-M1.4-S1.1:S1.2:S1.3-JOIN-M1.5-Po1.1-M1.1-M1.2-M1.3-Po1.2	

decide_priority(*node_i, *a, *b) routine. The stack *a* contains the *node_{ID}* of the nodes already traversed. The array *b*[] for any node gives the number of sub flow, alternate flow and main flow nodes encountered while traversing from the node *n* to that node. The cell *b*[1], *b*[2] and *b*[3] of the array *b*[] gives the number of sub flow, alternate flow and main flow nodes respectively. The last cell *b*[4] contains the sum of *b*[1], *b*[2] and *b*[3]. When the leaf node of a path or the last node of the loop is reached the array *b* is returned. Now, priority is decided as per above said scheme. When the node with *node_{ID}* = FORK is encountered (i.e. beginning of fork/join structure), we begin breadth-first search till we reach the node with *node_{ID}* = JOIN (i.e. the end of the fork/join structure). This is done so the path after the sub flows is not taken into account more than once, which would have been the case had we stuck with normal depth first search. The algorithm for assigning priorities among the generated test cases

Algorithm: *decide_Priority*(*node, *a, *b) do
 Declare *event_{id}*; // stores the identification tags
 Declare *flag*;=false; // Boolean variable to store true if a loop has been detected in the activity diagram else store false
event_{id}:= (node → next) → *node_{ID}*;
 for i=0 to a[i] ≠ NULL step 1 do // to detect loops (if any) in the activity diagram
 if *event_{id}* == a[i] then do
flag:= true;
 break;
 end
 end
 while node → next ≠ NULL and *flag* == false do // calculate the number of alternate or sub flow or main flows encountered
b[1]:= *b*[1]+length(node → s); // number of sub flow edges encountered

b[2]:= *b*[2]+length(node → a); // number of alternate flow encountered
b[3]:= *b*[3]+length(node → x)+1; // number of main flow encountered
a[++i]:= node → *node_{ID}*;
 for ∀ node arising from a node do
decide_Priority (*node_i*, a, b);
 end
 if node → next == NULL or *flag* == true then
b[4]:= *b*[1]+*b*[2]+*b*[3];
 return *b*;
 end

To prioritize the test sequences for the TDD, we found that there is no different path exists in the activity diagrams, as the flow of events in the activity diagrams are sequentially performed by the actors. In Figure 4, two paths encounters from M1.3 - A1.2 and A1.3. This node is the conditional node which is decided on the basis of the incident priority (higher or lower). Node A1.2 has 10 nodes (ns=3, na=0, nx=7) and A1.3 has 10 nodes (ns=3, na=0, nx=7). Both the paths have same complexity and hence no need to prioritize because at the time of development both the functionalities will be implemented simultaneously.

In order to more explicitly illustrate the applicability of proposed approach towards the test case prioritization, we consider a part of use case diagram of ATM system as an example (see Figure 6); the use case *Withdraw Cash* (*X*) and *Deposit Funds* (*Y*) are responsible for the withdraw and deposit functionalities [8]. The common part of both the use cases is *Validate PIN* (*Z*) functionality.

The activity diagram generated corresponding to an actor *Customer* is shown in Figure 7, where the first part shows the functionality of *Validate PIN* use case, second part shows the functionality of *Withdraw Cash* use case and last part shows the functionality of *Deposit Funds* use case.

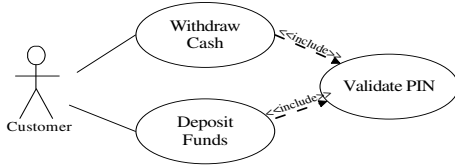


Figure 6: Part of use case diagram

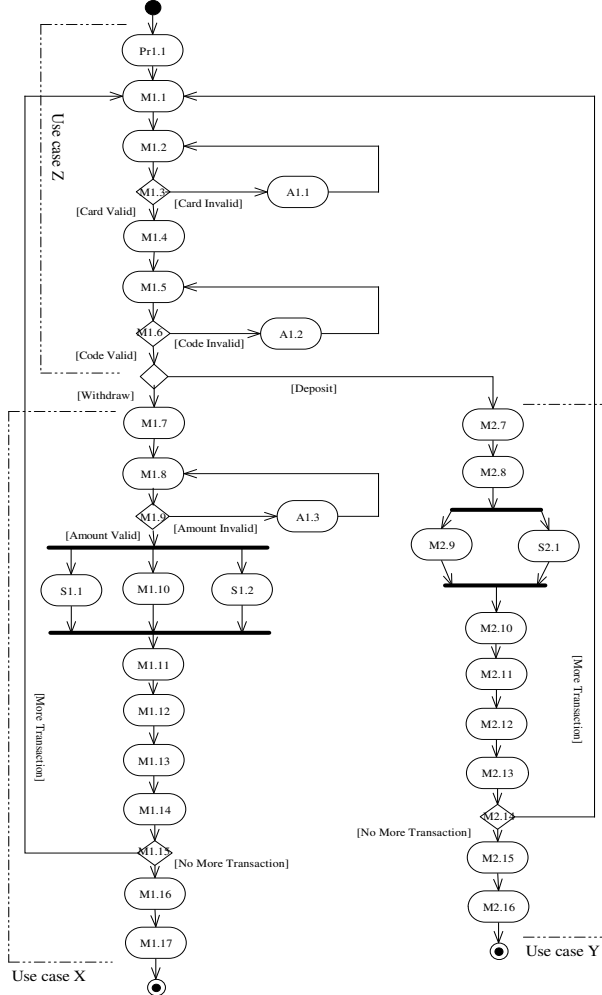


Figure 7: Actor-oriented activity diagram

Next, from the above actor-oriented activity diagram we generate the test sequences. Table 5 shows all the test sequences generated from the integrated activity diagram.

We have to prioritize the test cases for the given activity diagram (see Figure 7) and here both the use cases *Withdraw Cash* and *Deposit Funds* have the same customer priority (i.e., One). After third test case is generated, we have to go to node A1.1 before reaching node M1.4 because clearly this path has zero nodes after node A1.1 which is the last node (definition of which is defined above). While the number of nodes after M1.4 is 32 ($ns=5$, $na=4$, $nx=23$). So the later part is more complex than the former one as per the test case prioritization procedure. Similarly, we have two paths from M1.6. This node is the last common node in the two use cases. As both the use case *Withdraw Cash* and *Deposit Funds* have the same customer priority, so we have to decide

Table 5: Test sequences generated from actor-oriented activity diagram

Test Sequences
Pr1.1-M1.1
Pr1.1-M1.1-M1.2
Pr1.1-M1.1-M1.2-M1.3
Pr1.1-M1.1-M1.2-M1.3-A1.1
Pr1.1-M1.1-M1.2-M1.3-A1.1-M1.3
Pr1.1-M1.1-M1.2-M1.3-M1.4
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6-A1.2
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6-A1.2-M1.6
...M1.6-M2.7
...M1.6-M2.7-M2.8
...M1.6-M2.7-M2.8-FORK
...M1.6-M2.7-M2.8-FORK-M2.9,S2.1-JOIN
...M1.6-M2.7-M2.8-FORK-M2.9,S2.1-JOIN-M2.10
...M1.6-M2.7-M2.8-FORK-M2.9,S2.1-JOIN-M2.10-M2.11
...M2.10-M2.11-M2.12
...M2.10-M2.11-M2.12-M2.13
...M2.10-M2.11-M2.12-M2.14
...M2.10-M2.11-M2.12-M2.14-M1.1
...M2.10-M2.11-M2.12-M2.14-M2.15
...M2.10-M2.11-M2.12-M2.14-M2.15-M2.16
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6-M1.7
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6-M1.7-M1.8
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6-M1.7-M1.8-M1.9
Pr1.1-M1.1-M1.2-M1.3-M1.4-M1.5-M1.6-M1.7-M1.8-M1.9-A1.3
...M1.6-M1.7-M1.8-M1.9-A1.3-M1.9
...M1.6-M1.7-M1.8-M1.9-FORK
...M1.6-M1.7-M1.8-M1.9-FORK-S1.1,M1.10,S1.2-JOIN
...M1.6-M1.7-M1.8-M1.9-FORK-S1.1,M1.10,S1.2-JOIN-M1.11
...M1.6-M1.7-M1.8-M1.9-FORK-S1.1,M1.10,S1.2-JOIN-M1.11-M1.12
...M1.9-FORK-S1.1,M1.10,S1.2-JOIN-M1.11-M1.12-M1.13
...M1.9-FORK-S1.1,M1.10,S1.2-JOIN-M1.11-M1.12-M1.13-M1.14
...FORK-S1.1,M1.10,S1.2-JOIN-M1.11-M1.12-M1.13-M1.14-M1.15
...FORK-S1.1,M1.10,S1.2-JOIN-M1.11-M1.12-M1.13-M1.14-M1.15-M1.1
...M1.11-M1.12-M1.13-M1.14-M1.15-M1.16
...M1.11-M1.12-M1.13-M1.14-M1.15-M1.16-M1.17

which path to take. Node M1.7 has 15 nodes ($ns=2$, $na=2$, $nx=11$) and after it, while M2.7 has 12 nodes ($ns=1$, $na=1$, $nx=10$). So the path after the node M2.7 is given higher priority (less complex).

3. ANALYSIS OF VARIOUS TEST GENERATION APPROACHES

In this section, we present a comparative study of the existing approaches proposed in the literature for generating the test cases from use case specification. Though, the existing approaches (e.g., [5][7][3][24]) do not explicitly provide the test cases suitable for the TDD for incremental coding, but still the system-level test cases can be obtained from them.

Some brief details about these approaches are: Fröhlich et al. [5], approach requires the use case textual descriptions to be transformed into a UML state chart and then shows how test suites with a given coverage level can be automatically generated from these statecharts. In [24], a scenario-based approach is detailed in order to systematically derive test cases for system testing by generating the state charts and a graphical notation called dependency charts. Hartmann et al. [7] proposed an approach of generating the test cases from the activity diagrams (obtained from the use case textual descriptions), where first pair of lanes shows the typical

Table 6: Analysis of the Number of Generated Test Sequences

<i>Problem Requirement</i>	Test Case Generation Approaches	Briand et al. [3]	Fröhlich et al. [5]	Ryser et al. [24]	Hartmann et al. [7]	Proposed Approach (I)	Proposed Approach (II)
	Models used for generating test requirements	Use case, Sequence Diagram	Statechart	Statechart, Dependency chart	Activity Diagram	Without capturing dependencies	Capturing dependencies
ATM System [8]	- Validate PIN				5		
	- Deposit Funds				12		
	- Withdraw Cash	43	30	32	13	32	37
FRIEND System [2]	- Report Emergency				10		
	- Open Incident				11		
	- Allocate Resources	39	26	30	5	30	34
POS System [15]	- Process Sale				18		
	- Handle Returns				7		
	- Print Receipt	39	30	32	5	32	33
<i>Test sequences</i>		121	86	94	86	94	104

test scenario, flow of interaction or happy path, and remaining lane depicts the alternate courses. Briand et al. [3] proposed an approach of generating the system-level test cases based on the refinement of UML use case specification into the sequence (or collaboration) diagrams.

We have taken three problem requirements to generate the test sequences. Table 6 shows the total number of test sequences generated by applying their proposed approaches over three problem set. This analysis shows that our approach can generate the test cases as large as obtained from the Briand et al. [3] approach. The difference between generated test sequences by the two approaches that the Briand’s approach can capture the dependencies between the use cases more effectively such as the dependencies belonging to different actors by modeling all the use cases in a dependency graph. Whereas, our approach is stick towards capturing the interaction flow of events belonging to the actors associated with the system. This help in generating the smaller independent test sequences to larger ones incrementally for TDD.

However, the approach presented by [3] is heavy-weight, as this includes the transformation of activity diagram to create sequence diagrams. These sequence diagrams are then converted into regular expressions to generate the test sequences. However, agile-based software development requires an approach to be light-weight, quick and have a potential to adopt the desired changes. Similar to this, our approach generates the smaller to larger test sequences incrementally along with their prioritization suitable for TDD.

4. DISCUSSIONS

Working on the norms of TDD, our credence is that we must begin coding with the least complex test case and gradually ante the complexity levels which in turn will be fulfilled by the generation of test cases in the same prioritized fashion. This is favorable for two reasons. Firstly, TDD aims at evolving the design with each iteration in a sequential order because inculcating changes in the pre-existing code for making it compatible with the newly added hefty code is much easier than reflecting changes in the complex code on addition of the less complex code. Therefore, if we first code for the most complex test case then the chances are high that we will be coding according to a pre-conceived design and our iteration will try to hold up the sanctity of that design paradigm. Secondly, it will lead to the generation of the minimal and non-redundant piece of code.

In this paper, the generation of an integrated activity diagram corresponding to all use cases with unique actors, adds significant advantage to model the use case specification. This reduces the task of developing a separate activity diagrams for each use case helpful to a large extent as a large problem is bound to be accompanied by large number of use cases but would have only a few actors associated with them. Moreover, this integrated activity diagram helps in specifying the dependency related issues by clearly defining the profound and accurate details about the inter-dependencies of various use cases. This also helps in stripping out those common components or the modules that make use of same flow of events but later use different use cases while combining them.

In order to apply our approach of generating activity diagram for each actor, one concern may be the priority or the preference of the use case that will be encountered first. But, the priority of the use case is a major concern when, one is interested in generating and prioritizing the test cases for software development at an early stage. One way to decide the priority of a use case as per the customer’s discretion and can be expatiated during requirement elicitation. The contention may arise when the customer may assign a high priority to a functionality that is more complex to implement than a less complex functionality. The complexity of a test case will be defined by the complexity of a path described in the activity diagram of each actor. Here, we describe an approach which generates a more efficient test suite in increasing complexity according to the customer priority. In case of conflict between the priority of two paths presented in the activity diagram, high priority is given to the path that is less complex. In this manner, we are able to provide a complete set of prioritized and efficient test suite for each of the actors involved in the system.

One can easily make out that the test sequences generated using our proposed approach are at a high level and need to be converted to the execution level test cases. The mechanism for converting the abstract test cases into the execution level test cases is kept out of scope of this paper. We also believe that this approach can potentially be automated as an algorithmic transformation of generating actor-oriented activity diagrams from the use case requirements are presented in [28].

The main contribution of this paper is to obtain and prioritize the test cases for incremental coding, which is a prerequisite for TDD. We argue that actor-oriented activity diagram are better artifact in order to understand the depen-

dencies between the use cases belonging to the same actor as well as the use cases belonging to different actors. At the same time, the approach can also be used for better change management to adopt the desired changes made at the use case level.

5. RELATED WORKS

Prioritizing requirements for the agile software development maximizes the quality of product delivered to the stakeholders and to accommodate the desired changes in the specification. Although, several methods for prioritizing the requirements have been proposed but, the research on prioritizing quality requirements is very limited [27]. Racheva et al. [22], presented the results of an exploratory cross-case studies conducted to assess the effectiveness of agile development process - on agile prioritization and delivery processes in eight different software organizations. They found that the explicit and fundamental assumptions of agile requirement prioritization techniques are not suitable for all kinds of agile projects. The assumptions are, first - the role of client in the creation process; second - the prioritization criterion; and third - the role of prioritization process for achieving the desired goals.

Jeffrey et al. [10] presented an approach of prioritizing test cases generated using the statement coverage criteria, while considering the 'number of statements executed' to influence the output produced by the test case. The set of these executed statements corresponding to the relevant slice is computed by executing the test case. The heuristic for prioritizing test cases assigns higher weight to a test case with larger number of statements in its relevant slice. No existing prioritization techniques addresses the problem of multiple cases with same weight values. The existing test case prioritization techniques use a random approach to resolve the aforementioned issues. Moreover, there is no prioritization technique exist that can handle multiple test suites - assumed that there is only a single suite.

Svensson et al. [27], conducted an study at 11 successful companies developing software intensive systems to investigate how that quality requirements are prioritized in practice. They found that ad-hoc prioritization and priority grouping of requirements are the dominant methods for prioritizing quality requirements. The results also show that it is common to use customer input as criteria for the prioritization but absence of any such criteria was also common. While the need to derive a prioritized test suite from the use case specifications is widely recognized in the literature [26][13], few approaches have been proposed to achieve this derivation. This may be due to the lack of an intermediate model that bridge the gap between the use case textual descriptions and the formal specifications, typically required to generate the test cases as highlighted by [23][6]. Other cause as described in [3] may be that the level of detail represented through this model is coarse.

For this, model-based testing approaches have been used to formalize the specifications in order to use them in various phases of the software development process, and the same has been used for test case generation, requirement prioritization, change management, design modeling and so on [3][13][12]. Ottensoosera et al. [17], conducted an experimental study to compare two different types of modeling notations - graphical and textual. This experiment analyze that how effective is a notation in perspective with the users

and how efficiently the information can be conveyed to the the writer's. The result of the study suggested that the textual notations were found significantly better than the graphical models in terms of the users' understanding of the desired specification. But, in case of the graphical model, the trained readers can only showed significant differences.

The authors in [3][6] gives an insight to model the requirement specific dependencies between the use cases. Ryser et al. [24] details a scenario-based approach in which scenarios are formalized using state charts or dependency charts. These dependency charts takes into account a lot of intricacies dealing with time dependencies and data resource dependencies along with the sequential dependencies. In order to generate the system level test cases from the requirements, the independent set of requirements that are functionally dependent must be captured consistently. This helps to increase the coverage of the derived test cases. Another vital contribution for the test case generation using use cases can be acknowledged in [3]. Where, one activity diagram per actor has been created depicting the sequential constraints for each use case for an actor by grouping the use cases into swim lanes. The activity diagram is then transformed into a directed graph from which the use case sequences were derived. The use cases were then elaborated to generate the sequence diagrams. These sequence diagrams are then converted into regular expressions.

Our approach seems to be similar to Briand's approach, but it differs in few aspects. Firstly, we differ in a way dependencies between the use cases are modeled. Secondly, the test criterion in [3] is based on the regular expressions extracted from the activity diagram. This criterion resulted into a large number of test cases but our goal is different as we are concerned with the generation of test cases for incremental code development and that too in a prioritized fashion. On the similar guidelines, our paper presents an algorithm in order to generate a prioritized test suite as it is capable of handling more intricacies involved in a complex system. Our approach also provides apt weightage to customer's priority, one of the key aspects of defining precedence of the functionalities of a system, as highlighted by [26].

6. CONCLUSIONS AND FUTURE WORK

This paper present a formal approach to generate and select test cases for test driven development. The approach makes use of an integrated activity diagram for each actor separately, which are then translated into the prioritized test cases for TDD. Particular care is devoted to defining the prioritization of the use cases, taking into account their relevance to the application and their complexity, simpler use cases being preferred if the relevance is the same.

We aim to develop a desired tool support for automatic construction of actor-oriented activity diagram from the use case specification. This would also facilitate the automatic generation of test requirements along with their prioritization.

7. REFERENCES

- [1] S. W. Ambler. The agile scaling model (ASM): adapting agile methods for complex environments. *Environments*, 2009.

- [2] B. Bruegge. *Object-Oriented Software Engineering: Using Uml, Patterns and Java*, Pearson, Second Edition edition, 2009.
- [3] L. C. Briand and Y. Labiche. A uml-based approach to system testing. In *Proceedings of the 4th International Conference on The Unified Modeling Language, Modeling Languages, Concepts, and Tools, UML'01*, pages 194–208, London, UK, Springer-Verlag, 2001.
- [4] D. Ince. *Object-Oriented Software Engineering with C++*, McGraw-Hill edition, 1991.
- [5] P. Fröhlich and J. Link. Automated test case generation from dynamic models. In *Proceedings of the 14th European Conference on Object-Oriented Programming, ECOOP '00*, pages 472–492, London, UK, Springer-Verlag, 2000.
- [6] P. Gibson *Formal Requirements Models: Simulation, Validation and Verification*, Technical report, Computer Science Dept., Nat'l Univ. of Maynooth, Ireland edition edition, 2001.
- [7] J. Hartmann, M. Vieira, H. Foster, and A. Ruder. A uml-based approach to system testing. *Innovations in Systems and Software Engineering*, 1(1):12–24, 2005.
- [8] IBM. *Object-Oriented Analysis and Design with UML2 and Rational Software Modeler*. IBM Corporation, 2006.
- [9] I. Jacobson, M. Christerson, P. Jonsson and G. Overgaard. *Object-Oriented Software Engineering: A Use-Case Driven Approach*, Addison-Wesley, Reading, MA, 1992 edition, 1992.
- [10] D. Jeffrey and N. Gupta. Experiments with test case prioritization using relevant slices. *Journal of System and Software*, 81(2):196–221, 2008.
- [11] J. Kanyaru, K. Phalp, K. Cox, and A. Jha. Supporting the consideration of dependencies in use case specifications. In *11th International Workshop on Requirements Engineering: Foundation For Software Quality-REFSQ05, Porto, Portugal*, pages 13–14. 2005.
- [12] M. Katara and A. Kervinen. Making model-based testing more agile: A use case driven approach. In E. Bin, A. Ziv, and S. Ur, editors, *Hardware and Software, Verification and Testing*, volume 4383 of *Lecture Notes in Computer Science*, pages 219–234. Springer Berlin Heidelberg, 2007.
- [13] H. Kim, S. Kang, J. Baik, and I. Ko. Test cases generation from UML activity diagrams. In *Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing, 2007. SNPD 2007. Eighth ACIS International Conference on*, volume 3, pages 556–561, 2007.
- [14] C. Larman. *Agile and Iterative Development: A Manager's Guide*. Pearson Education, 2003.
- [15] C. Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3/e*. Pearson Education India, 2012.
- [16] C. Nebut, F. Fleurey, Y. Le Traon, and J.-M. Jezequel. Automatic test generation: A use case driven approach. *IEEE Transactions on Software Engineering*, 32(3):140–155, 2006.
- [17] A. Ottensooser, A. Fekete, H. A. Reijers, J. Mendling, and C. Menictas. Making sense of business process descriptions: An experimental comparison of graphical and textual notations. *Journal of System and Software*, 85(3):596–606, 2012.
- [18] B. Paech. On the role of activity diagrams in UML - a user task centered development process for uml. In *Selected Papers from the First International Workshop on The Unified Modeling Language UML'98: Beyond the Notation*, pages 267–277, London, UK, Springer-Verlag, 1998.
- [19] M. Pančur and M. Ciglaric. Impact of test-driven development on productivity, code and tests: A controlled experiment. *Information and Software Technology*, 53(6):557–573, 2011.
- [20] T. Quatrani *Visual Modeling with Rational Rose and UML*, Reading, Massachusetts, Addison Wesley edition, 1998.
- [21] R. V. Binder. *Testing Object-Oriented Systems - Models, Patterns, and Tools*, Addison-Wesley edition, 1999.
- [22] Z. Racheva, M. Daneva, K. Sikkil, A. Herrmann, and R. Wieringa. Do we know enough about requirements prioritization in agile projects: Insights from a case study. In *Proceedings of the 2010 18th IEEE International Requirements Engineering Conference, RE '10*, pages 147–156, Washington, USA, IEEE Computer Society, 2010.
- [23] B. Regnell, K. Kimbler, and A. Wesslen. Improving the use case driven approach to requirements engineering. In *Requirements Engineering, 1995., Proceedings of the Second IEEE International Symposium on*, pages 40 – 47, 1995.
- [24] J. Ryser and M. Glinz. A scenario-based approach to validating and testing software systems using statecharts. In *In 12th International Conference on Software and Systems Engineering and their Applications (ICSSEA'99)*, page 7, 1999.
- [25] M. Sarma, D. Kundu, and R. Mall. Automatic test case generation from UML sequence diagram. In *Advanced Computing and Communications, 2007. ADCOM 2007. International Conference on*, pages 60–67, 2007.
- [26] H. Srikanth and L. Williams. On the economics of requirements-based test case prioritization. *SIGSOFT Software Engineering Notes*, 30(4):1–3, 2005.
- [27] R. Svensson, T. Gorschek, B. Regnell, R. Torkar, A. Shahrokni, R. Feldt, and A. Aurum. Prioritization of quality requirements: State of practice in eleven companies. In *Requirements Engineering Conference (RE), 2011 19th IEEE International*, pages 69–78, 2011.
- [28] S. Tiwari, S. S. Rathore, A. Singh, A. Singh, and A. Gupta. An approach to generate actor-oriented activity charts from use case requirements. In *Proceedings of the 2012 19th Asia-Pacific Software Engineering Conference - Volume 01, APSEC '12*, pages 270–275, Washington, USA, IEEE Computer Society, 2012.
- [29] H. Zhu, P. A. V. Hall, and J. H. R. May. Software unit test coverage and adequacy. *ACM Computing Survey*, 29(4):366–427, 1997.